WHY THE CHECK CAN BE CHEAP

Proof that travels

Checking a reactor-physics number should not cost what producing it did. SMRFORGE seals every result so a reviewer can confirm it was not touched and re-run only the cheap part to see it land where it should. The same seals let an agent carry a whole pipeline forward, one checkable step at a time.

01 · A CHECK THAT COSTS ALMOST NOTHING

You should not re-run the cluster to confirm a number.

A full reproducibility run re-executes the entire pipeline to re-check one result. For a continuous-energy Monte Carlo core that is hours of compute on a cluster, every time anyone wants to be sure. SMRFORGE replaces most of that with two moves a reviewer can make alone.

`smrforge verify <bundle>`

Verify, offline, in milliseconds

Recompute the fingerprints and the signature on your own machine. No engine, no network, no cluster. It confirms the number was not altered since it was sealed, and names the exact section if it was.

`smrforge reproduce <bundle>`

Re-run only the cheap step

A result built from sealed upstream constants re-runs only the downstream deterministic step on those already-verified constants. You check the expensive Monte Carlo once, then never redo it to confirm the numbers that flow from it.

FASTER

Milliseconds, not hours

Verify is a hash check and a signature check. Reproduce re-runs a deterministic solve, not a Monte Carlo campaign.

CHEAPER

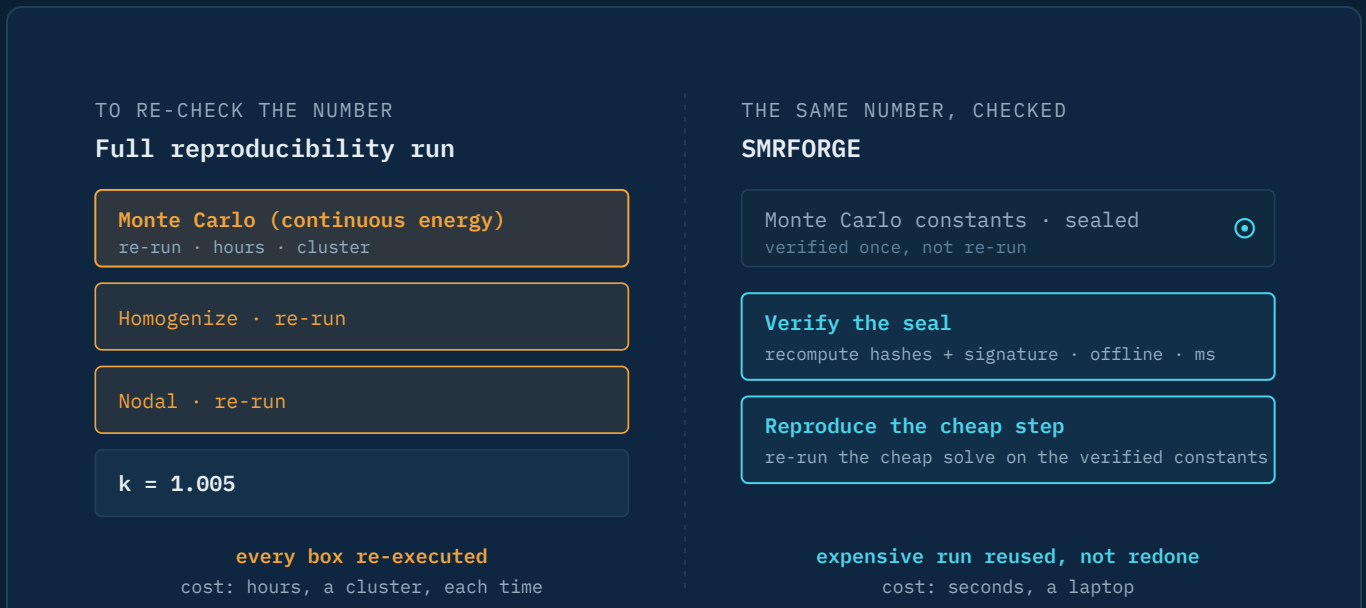
A laptop, not a cluster

The expensive run happens once. Every later confirmation is work a reviewer already has the hardware for.

SMARTER

No trust required

The tolerance is written down before the run. The check stands on its own, offline, without taking the producer's word.



Re-checking one number. Left, redo the whole pipeline. Right, verify the seal and re-run only the cheap step on constants that were already checked. Illustrative.

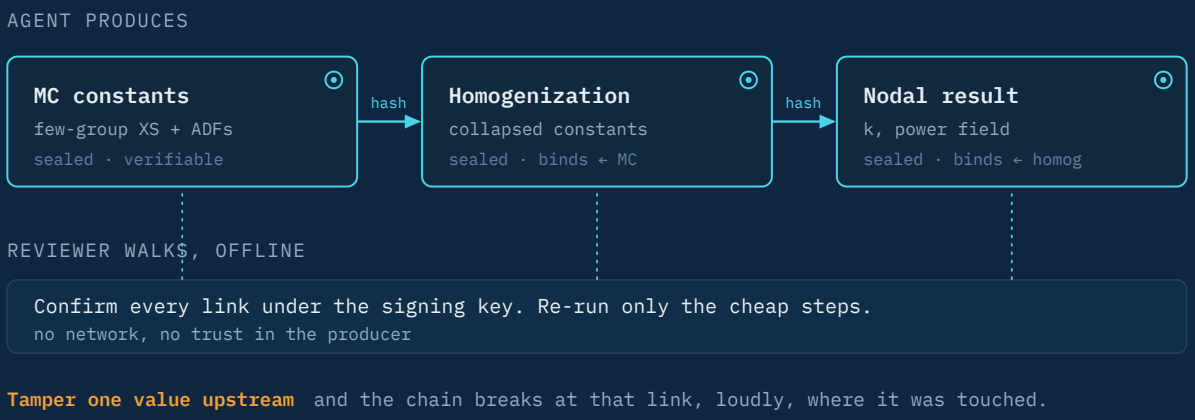
Said plainly. Verify proves custody, not correctness. The original run is still the authority on the physics. SMRFORGE makes the checking cheap, not the computing.

02 · A CHAIN BUILT FOR AGENTS

Each step is a file an agent produces and another can check.

A sealed result is just a signed file. A result cites the one it was built from by content hash, so a pipeline becomes a chain of sealed steps, each bound to the last. That shape is exactly what an autonomous pipeline needs.

An agent runs a step, seals it, and binds it to the upstream result. A reviewer, human or agent, walks that chain offline, confirms every link under the signing key, and re-runs only the cheap steps. Change one number upstream and the chain breaks exactly where it was touched.



One result cites the previous by content hash. Agents produce the chain, a reviewer walks it offline. Illustrative.

What is real today. The chaining is live now. A real homogenization-to-nodal chain verifies end to end under a signing key. The autonomous orchestration across a full pipeline is where this is going. AI can produce the number. SMRFORGE is what makes it defensible.

SMRFORGE results are screening-to-reference grade and advisory. They are checked against public criticality benchmarks on open ENDF/B-VIII data, and reproducible by anyone, a lab included. It is not a licensing code, and it says so on every result.

Verify and tamper a real sealed result → smrforge.io/check.html

Break the Seal → smrforge.io/break-the-seal.html

Chris Whalen chris@smrforge.io · smrforge.io